LAPORAN
WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)
SEMESTER III



Acara 20

Nama:

Nila Agustin Prima Aurelia

NIM:

E41250666

Golongan:

A

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

BAB 1. PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan aplikasi berbasis web, keamanan menjadi salah satu aspek penting yang perlu diperhatikan. Salah satu hal yang harus diterapkan adalah autentikasi untuk memastikan bahwa pengguna yang mengakses sistem merupakan pengguna yang memiliki hak akses.

JSON Web Token (JWT) merupakan salah satu metode yang dapat digunakan untuk melakukan autentikasi. JWT memungkinkan server memberikan token kepada pengguna setelah proses login berhasil. Token tersebut kemudian digunakan untuk mengakses endpoint yang membutuhkan autentikasi tanpa harus mengirimkan username dan password secara berulang.

Selain autentikasi, penanganan error juga diperlukan agar aplikasi dapat memberikan respons yang sesuai ketika terjadi kesalahan, seperti data yang tidak ditemukan, token yang tidak valid, atau pengguna yang tidak memiliki izin akses. Oleh karena itu, praktikum ini dilakukan untuk memahami penerapan JWT, middleware autentikasi, serta penanganan error dalam pengembangan RESTful API

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep JSON Web Token (JWT).
2. Memahami proses autentikasi pengguna menggunakan token.
3. Mengetahui cara membuat dan memvalidasi token JWT.
4. Memahami penggunaan middleware untuk membatasi akses endpoint.
5. Memahami penerapan penanganan error pada RESTful API.
6. Menguji respons API ketika proses autentikasi berhasil maupun gagal.

BAB 2. DASAR TEORI

2.1 JSON Web Token (JWT)

JSON Web Token (JWT) adalah standar terbuka yang digunakan untuk menyampaikan informasi antar pihak dalam bentuk objek JSON. Informasi tersebut dapat diverifikasi menggunakan tanda tangan digital sehingga penerima dapat memastikan bahwa token berasal dari pihak yang benar dan tidak mengalami perubahan.

JWT sering digunakan dalam proses autentikasi RESTful API. Setelah pengguna berhasil melakukan login, server akan menghasilkan token yang dapat digunakan pengguna untuk mengakses endpoint tertentu.

2.2 Struktur JWT

JWT terdiri dari tiga bagian utama, yaitu:

1. Header, berisi informasi mengenai jenis token dan algoritma yang digunakan untuk membuat tanda tangan.
2. Payload, berisi data atau klaim, misalnya identitas pengguna dan waktu kedaluwarsa token.
3. Signature, digunakan untuk memverifikasi keaslian token dan memastikan bahwa bagian token yang ditandatangani tidak mengalami perubahan.

Ketiga bagian tersebut dipisahkan menggunakan tanda titik (.).

2.3 Proses Autentikasi Menggunakan

Proses autentikasi menggunakan JWT dimulai ketika pengguna mengirimkan username dan password melalui endpoint login. Server kemudian memeriksa apakah data tersebut sesuai dengan data pengguna yang tersimpan.

Jika autentikasi berhasil, server membuat JWT dan mengirimkannya kepada pengguna. Pengguna selanjutnya mengirimkan token tersebut pada header Authorization ketika mengakses endpoint yang membutuhkan autentikasi.

Server memvalidasi token sebelum memberikan akses terhadap data yang diminta. Jika token valid, permintaan dapat diproses. Sebaliknya, jika token tidak valid atau sudah kedaluwarsa, server akan memberikan respons error yang sesuai.

2.4 Middleware Autentikasi

Middleware merupakan komponen yang digunakan untuk memeriksa permintaan sebelum diteruskan ke proses utama aplikasi. Dalam autentikasi JWT, middleware berfungsi untuk memeriksa keberadaan dan validitas token.

Jika token valid, permintaan diteruskan ke endpoint yang dituju. Jika token tidak tersedia atau tidak valid, middleware akan menghentikan permintaan dan mengirimkan respons yang sesuai.

2.5 Penanganan Error pada RESTful API

Penanganan error merupakan proses mengelola kesalahan yang terjadi ketika API menerima atau memproses permintaan. Respons error harus memberikan informasi yang jelas mengenai kondisi permintaan tanpa membocorkan informasi internal sistem.

Beberapa kode status HTTP yang umum digunakan adalah:

- 200 OK: Permintaan berhasil diproses.
- 201 Created: Data berhasil dibuat.

- 400 Bad Request: Permintaan tidak sesuai dengan ketentuan.
- 401 Unauthorized: Autentikasi tidak berhasil atau token tidak valid.
- 403 Forbidden: Pengguna tidak memiliki izin untuk mengakses sumber daya.
- 404 Not Found: Sumber daya yang diminta tidak ditemukan.
- 500 Internal Server Error: Terjadi kesalahan pada server.

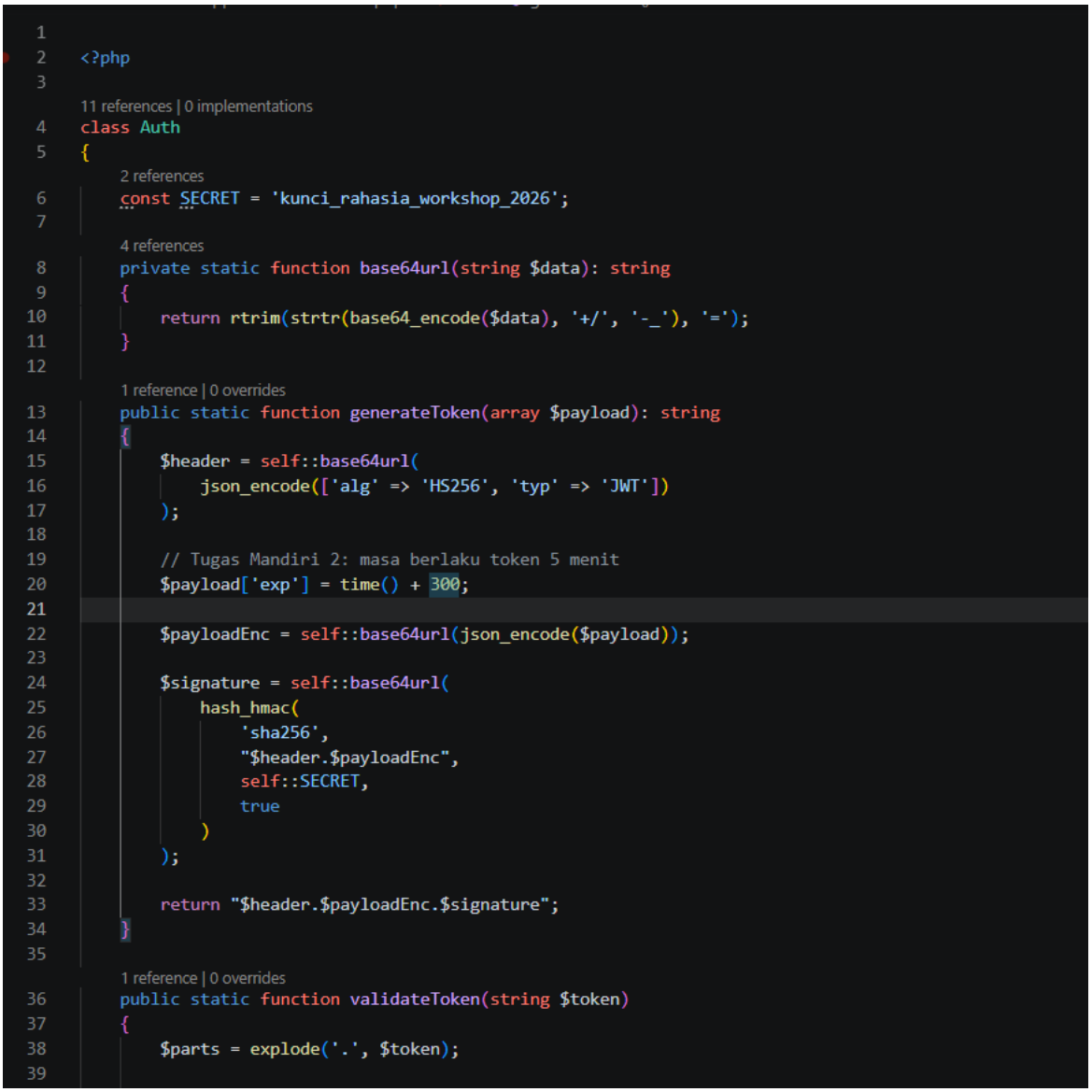
Penggunaan kode status HTTP yang sesuai membantu klien memahami hasil permintaan yang dikirimkan kepada API.

BAB 3. HASIL PRAKTIKUM

3.1 Prosedur Kerja

Langkah 1. Membuat Class Auth untuk JWT

Langkah pertama adalah membuat class Auth pada file app/core/Auth.php. Class ini digunakan untuk menangani proses pembuatan dan validasi token JWT. Token dibuat menggunakan algoritma HS256, secret key kunci_rahasia_workshop_2026, serta waktu kedaluwarsa selama satu jam (exp = time() + 3600).



```php

<?php

11 references | 0 implementations
class Auth
{
    2 references
    const SECRET = 'kunci_rahasia_workshop_2026';

    4 references
    private static function base64url(string $data): string
    {
        return rtrim(strtr(base64_encode($data), '+/', '-_'), '=');
    }

    1 reference | 0 overrides
    public static function generateToken(array $payload): string
    {
        $header = self::base64url(
            json_encode(['alg' => 'HS256', 'typ' => 'JWT'])
        );

        // Tugas Mandiri 2: masa berlaku token 5 menit
        $payload['exp'] = time() + 300;

        $payloadEnc = self::base64url(json_encode($payload));

        $signature = self::base64url(
            hash_hmac(
                'sha256',
                "$header.$payloadEnc",
                self::SECRET,
                true
            )
        );

        return "$header.$payloadEnc.$signature";
    }

    1 reference | 0 overrides
    public static function validateToken(string $token)
    {
        $parts = explode('.', $token);

```

Langkah 2. Menambahkan Fungsi Login

Langkah selanjutnya adalah menambahkan fungsi login() untuk memproses autentikasi pengguna. Fungsi ini memeriksa data login yang dikirimkan, kemudian menghasilkan token JWT apabila autentikasi berhasil. Token tersebut digunakan sebagai bukti autentikasi ketika pengguna mengakses endpoint yang dilindungi.

```php
11 references | 0 overrides
    public function login(): void
    {
        $data = json_decode(file_get_contents('php://input'), true);
        $data = is_array($data) ? $data : [];

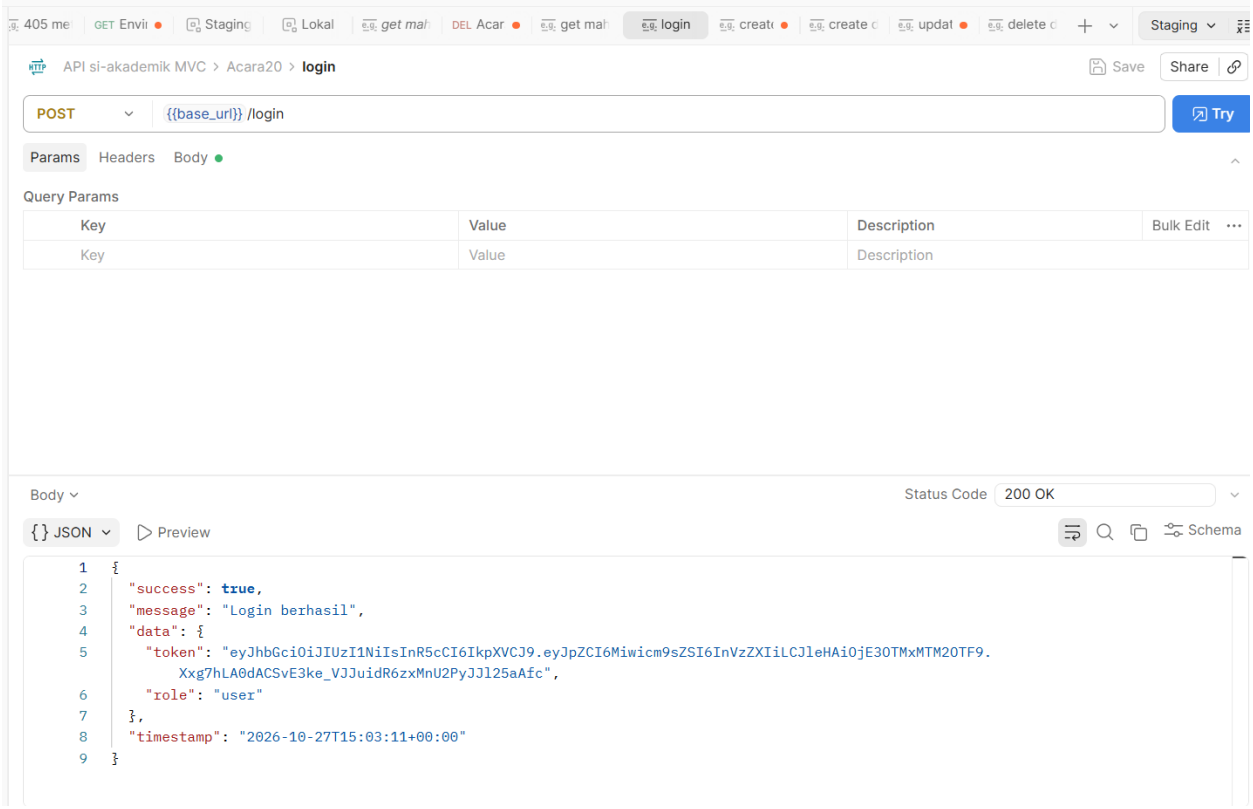
        $user = $this->model->findByUsername(
            $data['username'] ?? ''
        );

        if (
            !$user ||
            !password_verify(
                $data['password'] ?? '',
                $user['password']
            )
        ) {
            Response::json(401, false, 'Username atau password salah');
        }

        $token = Auth::generateToken([
            'id' => $user['id'],
            'role' => $user['role']
        ]);

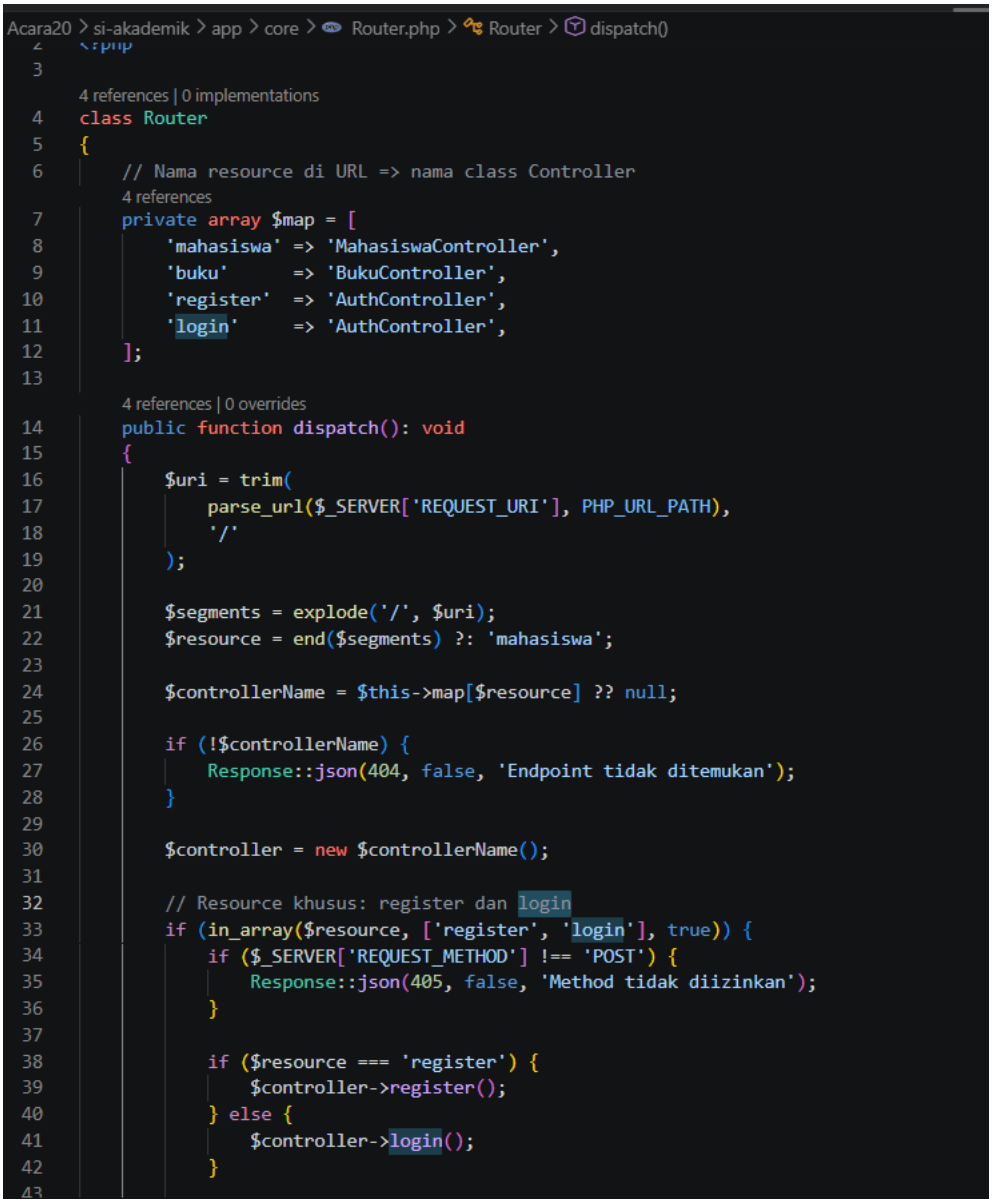
        Response::json(
            200,
            true,
            'Login berhasil',
            [
                'token' => $token,
                'role' => $user['role']
            ]
        );
    }

}
```



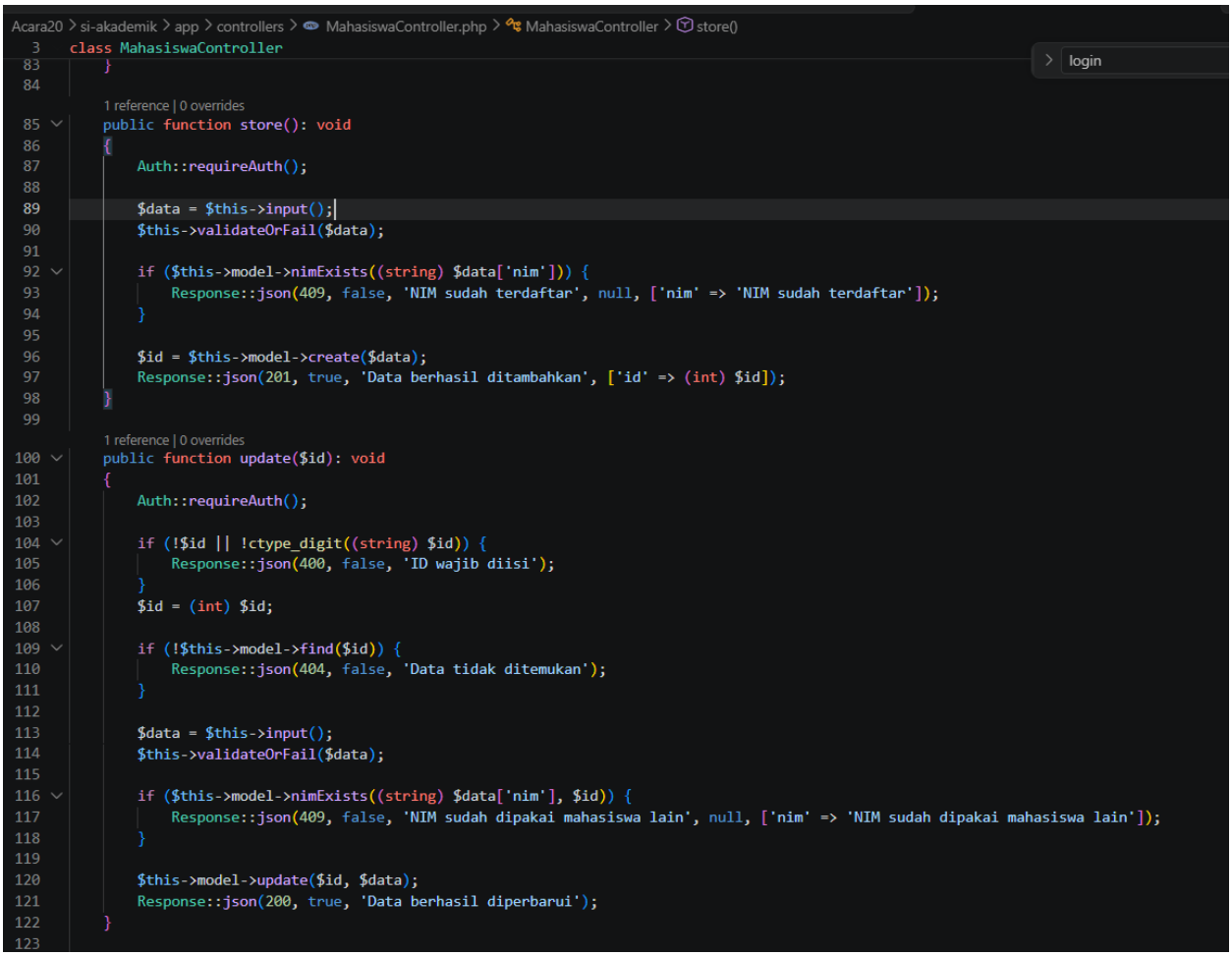
Langkah 3. Menambahkan Route Login

Route login ditambahkan agar pengguna dapat mengakses proses autentikasi melalui endpoint yang telah ditentukan. Endpoint ini menerima permintaan login dan menjalankan fungsi login() untuk memeriksa data pengguna serta menghasilkan token.



Langkah 4. Melindungi Endpoint Mahasiswa

Autentikasi diterapkan pada fungsi store(), update(), dan destroy() di bagian Mahasiswa. Ketiga fungsi tersebut digunakan untuk menambah, memperbarui, dan menghapus data mahasiswa sehingga aksesnya perlu dilindungi menggunakan token JWT.Sementara itu, fungsi index() tetap dapat diakses tanpa autentikasi sesuai ketentuan pada modul.



Langkah 5. Menambahkan Penanganan Error

Penanganan error diterapkan menggunakan try-catch pada bagian model untuk menangani kesalahan yang terjadi ketika aplikasi memproses permintaan atau berinteraksi dengan database. Dengan penanganan tersebut, kesalahan dapat dikelola sehingga aplikasi tidak langsung berhenti ketika terjadi masalah dalam proses pengolahan data.



```php
4 references | 0 overrides
public function create(array $data): string
{
    try {
        $stmt = $this->db->prepare(
            'INSERT INTO mahasiswa
                (nim, nama, email, prodi_id, angkatan, status)
             VALUES
                (:nim, :nama, :email, :prodi_id, :angkatan, :status)'
        );

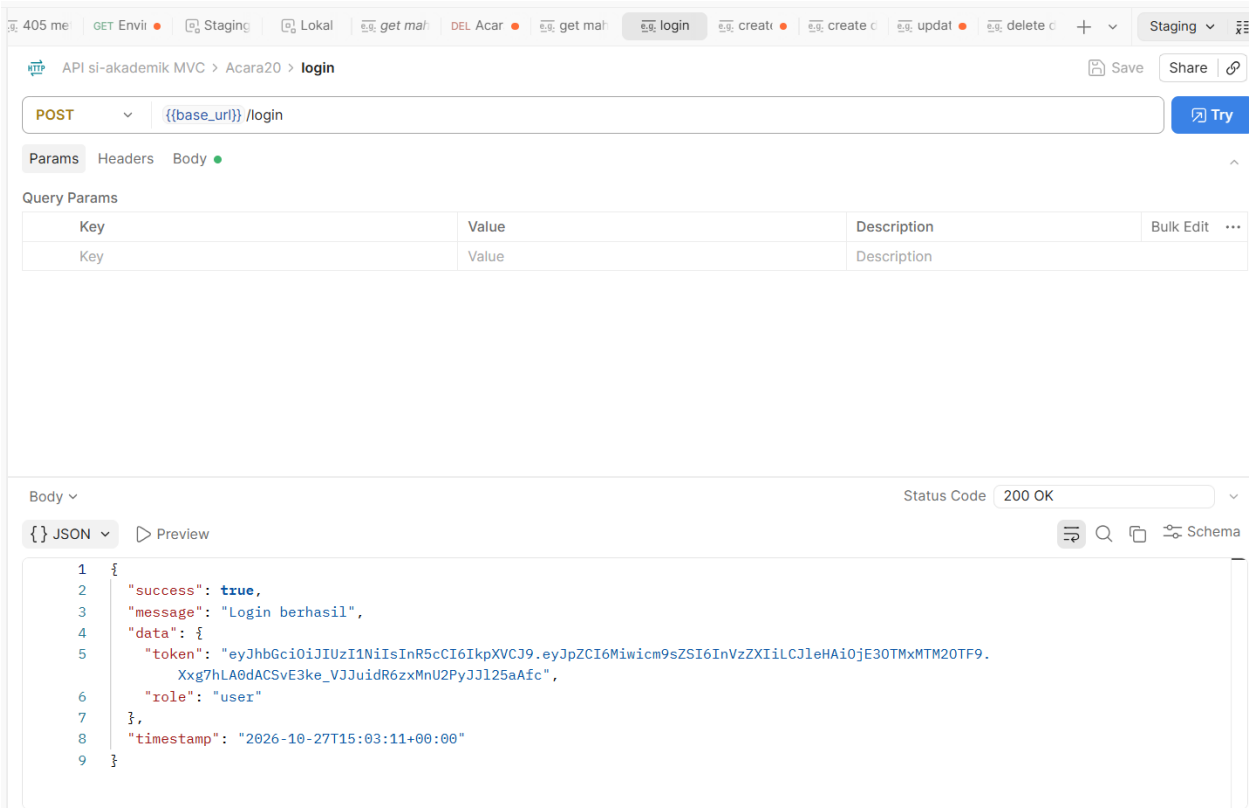
        $stmt->execute([
            ':nim'      => $data['nim'],
            ':nama'     => $data['nama'],
            ':email'    => $data['email'],
            ':prodi_id' => $data['prodi_id'],
            ':angkatan' => $data['angkatan'],
            ':status'   => $data['status'] ?? 'aktif',
        ]);

        return $this->db->lastInsertId();
    } catch (PDOException $e) {
        Response::json(
            500,
            false,
            'Terjadi kesalahan pada server'
        );
    }
}
```

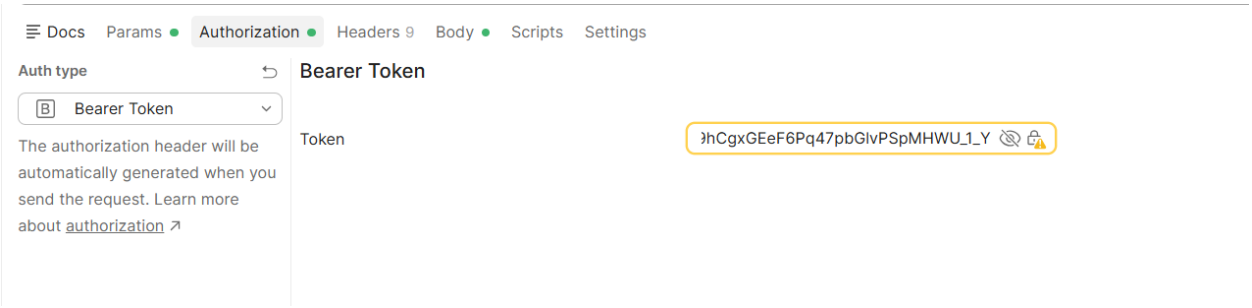
Langkah 6. Menguji Autentikasi JWT

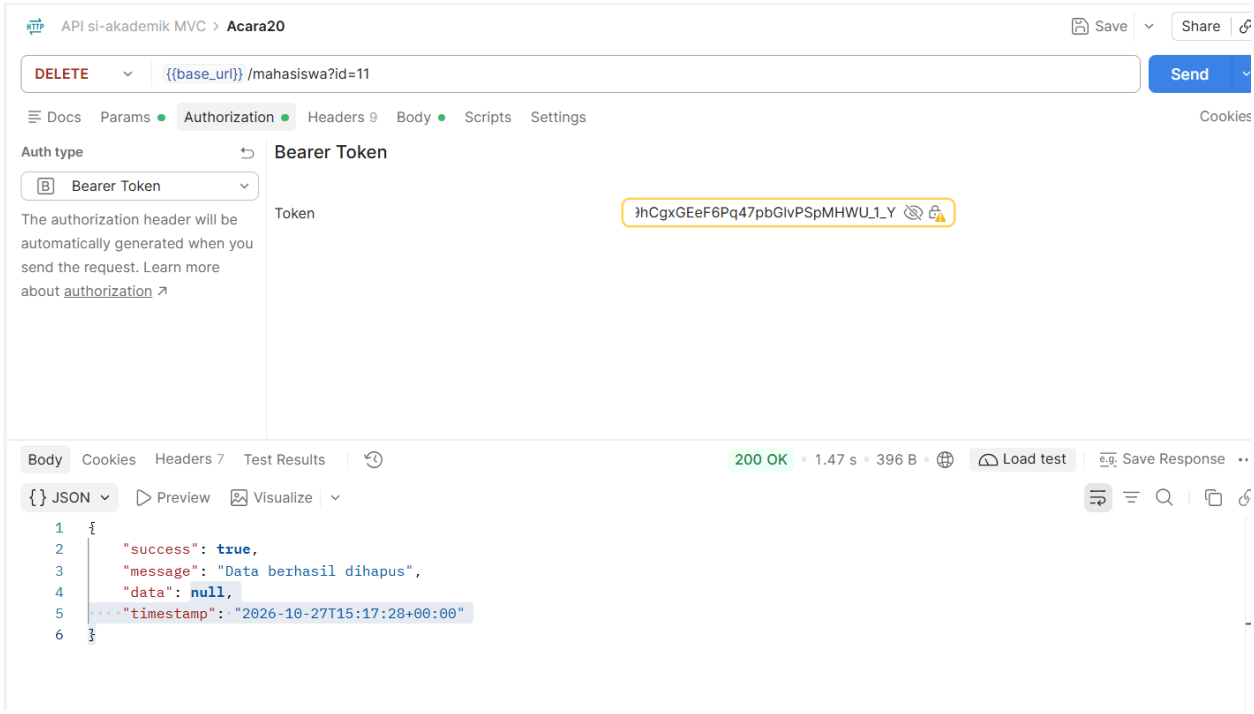
Pengujian dilakukan menggunakan Postman dengan beberapa skenario berikut:

1. Melakukan login untuk memperoleh token JWT.

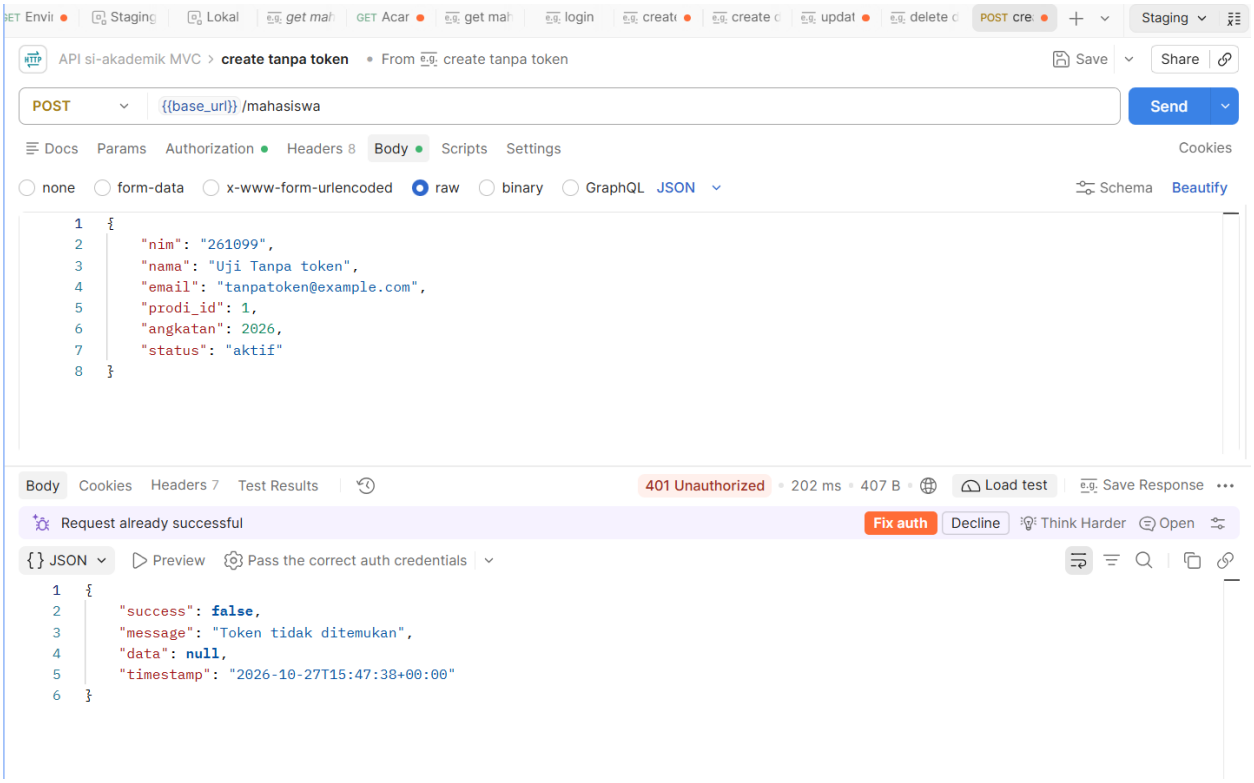


2. Mengakses endpoint yang dilindungi dengan menyertakan token pada header Authorization menggunakan format Bearer token.

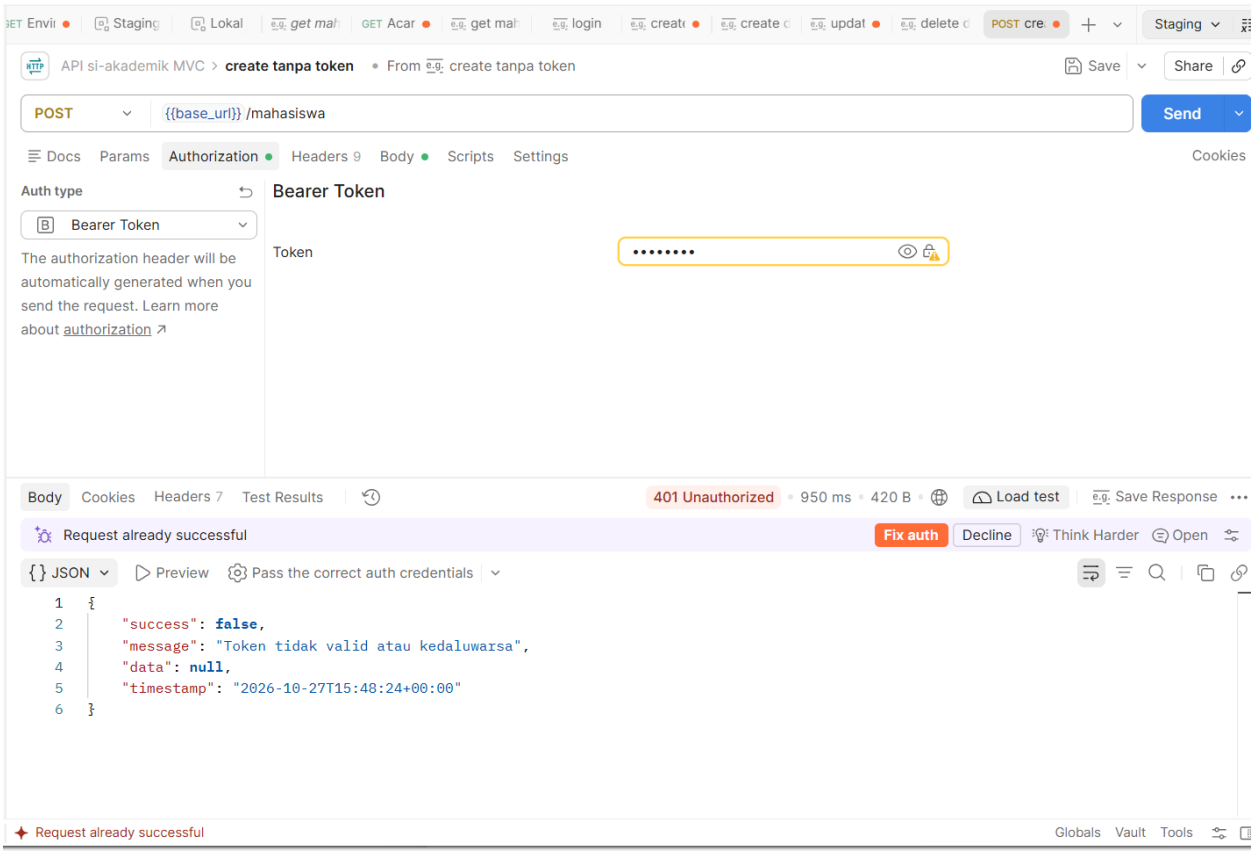




3. Mengakses endpoint yang dilindungi tanpa menyertakan token.



4. Mengakses endpoint menggunakan token yang telah diubah atau dimanipulasi.



Pengujian ini digunakan untuk memastikan bahwa endpoint yang dilindungi hanya dapat diakses menggunakan token yang valid. Permintaan tanpa token atau dengan token yang telah dimanipulasi seharusnya ditolak

3.2 Tugas Praktik Mandiri

Tugas 1. Analisis Kelemahan JWT Stateless

JWT memiliki sifat stateless, yaitu server tidak harus menyimpan sesi autentikasi pengguna di dalam database atau penyimpanan sesi server. Informasi autentikasi disampaikan melalui token yang dibawa pengguna setiap kali mengakses endpoint yang dilindungi.

Sifat tersebut membuat JWT praktis digunakan pada RESTful API. Namun, terdapat kelemahan ketika pengguna ingin melakukan logout. Karena server tidak menyimpan status sesi token secara langsung, token yang sudah diterbitkan masih dapat digunakan sampai masa berlakunya habis, selama token tersebut masih valid dan tidak ada mekanisme tambahan yang membatalkannya.

Salah satu cara untuk mengatasi kelemahan tersebut adalah menggunakan token blacklist. Token yang telah dinyatakan tidak berlaku dimasukkan ke dalam daftar blokir, kemudian server memeriksa daftar tersebut ketika menerima permintaan. Cara ini memerlukan penyimpanan tambahan dan mengurangi sifat stateless karena server perlu memeriksa status token.

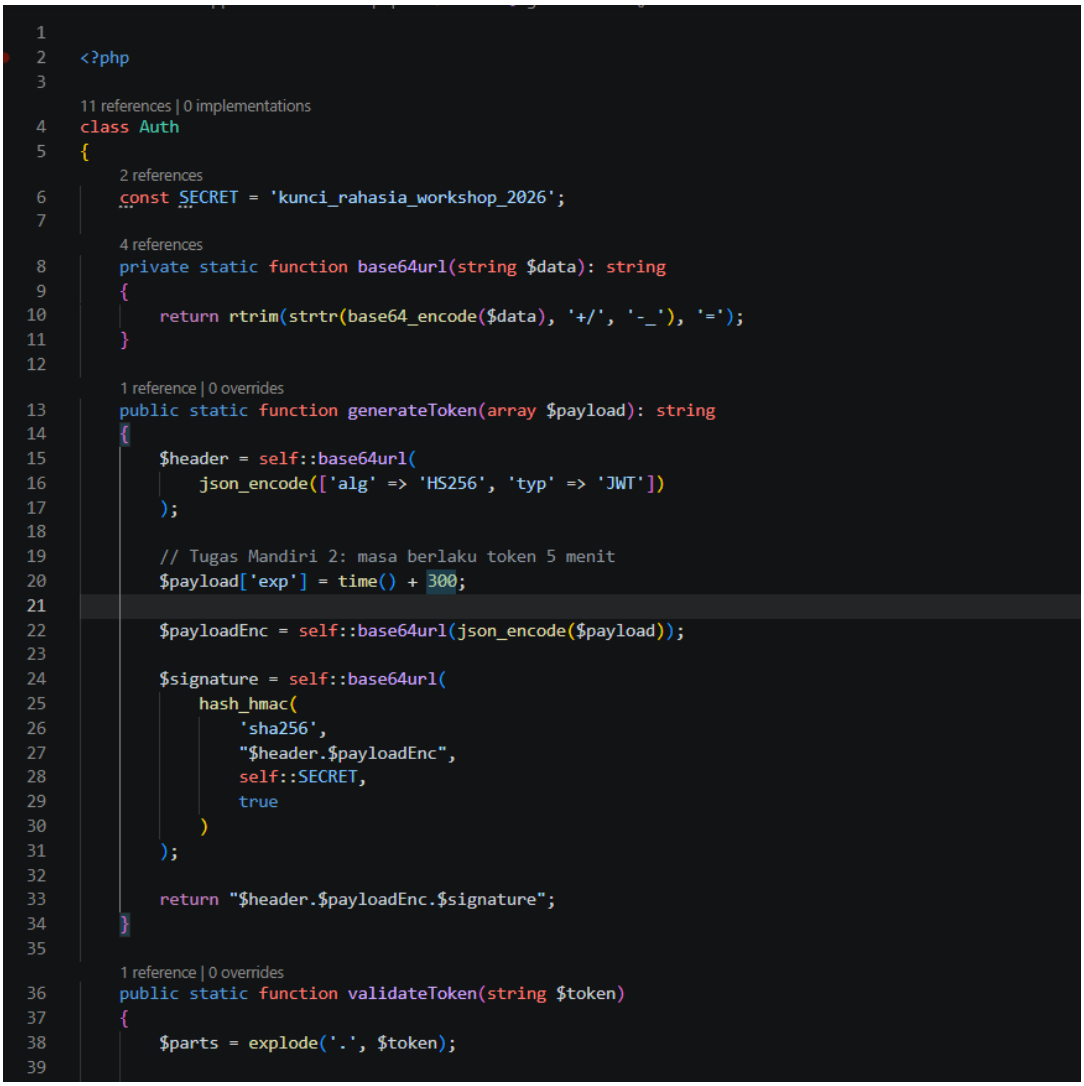
Dapat ditarik Kesimpulan bahwa JWT memudahkan autentikasi tanpa penyimpanan sesi pada server, tetapi logout tidak otomatis membatalkan token yang sudah diterbitkan. Oleh karena itu, diperlukan mekanisme tambahan seperti token blacklist apabila token harus dapat dibatalkan sebelum masa berlakunya habis.

Tugas 2. Memperpendek Masa Berlaku Token

Pada tugas mandiri ini, masa berlaku token JWT diubah menjadi lima menit. Perubahan dilakukan pada fungsi Auth::generateToken() dengan mengatur nilai waktu kedaluwarsa menjadi:

exp = time() + 300

Nilai 300 merupakan durasi 300 detik atau lima menit. Dengan perubahan ini, token hanya berlaku selama lima menit sejak diterbitkan. Masa berlaku token JWT menjadi lebih singkat. Apabila waktu kedaluwarsa telah terlewati, token seharusnya ditolak ketika digunakan untuk mengakses endpoint yang dilindungi.



```php

<?php

11 references | 0 implementations
class Auth
{
    2 references
    const SECRET = 'kunci_rahasia_workshop_2026';

    4 references
    private static function base64url(string $data): string
    {
        return rtrim(strtr(base64_encode($data), '+/', '-_'), '=');
    }

    1 reference | 0 overrides
    public static function generateToken(array $payload): string
    {
        $header = self::base64url(
            json_encode(['alg' => 'HS256', 'typ' => 'JWT'])
        );

        // Tugas Mandiri 2: masa berlaku token 5 menit
        $payload['exp'] = time() + 300;

        $payloadEnc = self::base64url(json_encode($payload));

        $signature = self::base64url(
            hash_hmac(
                'sha256',
                "$header.$payloadEnc",
                self::SECRET,
                true
            )
        );

        return "$header.$payloadEnc.$signature";
    }

    1 reference | 0 overrides
    public static function validateToken(string $token)
    {
        $parts = explode('.', $token);

```

BAB 4. KESIMPULAN

Berdasarkan praktikum yang dilakukan, dapat disimpulkan bahwa JSON Web Token (JWT) dapat digunakan sebagai mekanisme autentikasi pada RESTful API. Token diberikan setelah pengguna berhasil melewati proses login dan digunakan kembali ketika mengakses endpoint yang membutuhkan autentikasi.

Middleware berperan dalam memeriksa validitas token sebelum permintaan diproses oleh endpoint. Selain itu, penanganan error diperlukan agar API dapat memberikan respons yang sesuai terhadap permintaan yang berhasil maupun yang mengalami kesalahan.

Dengan memahami penerapan JWT, middleware, dan penanganan error, pengembang dapat membangun RESTful API dengan mekanisme autentikasi dan pengelolaan permintaan yang lebih terstruktur.